Set_A_Lab10 - Results

Attempt 1 of 1

Written Apr 1, 2026 11:35 AM - Apr 1, 2026 12:16 PM

Question 1

The goal of this question is to become familiar with some fundamental methods of communication between processes (or IPC). Sender (*sender.c*) and receiver (*reciever.c*) are given as follows (use online Linux to run your code).

sender.c

```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>
#include <sys/msg.h>
#define MAX_TEXT 512
struct my_msg_st
{
    long int my_msg_type;
    char some_text[MAX_TEXT];
};
int number_message = 5;
int main(int argc, char *argv[]){
    key_t key = 1234;
    struct my_msg_st some_data;
    int msgid;
    char buffer[BUFSIZ];
    if (argc == 2)
        number_message = atoi(argv[1]);
    msgid = msgget(key, 0666 | IPC_CREAT);
    if (msgid == -1){
        fprintf(stderr, "msgget failed with error: %d\n", errno);
        exit(EXIT_FAILURE);
    }
    for (int msg_itr = 1; msg_itr <=number_message; msg_itr++){
        some_data.my_msg_type = 1;
        if (msg_itr == (number_message)){
                sprintf(some_data.some_text, "end\n");
                printf("Message [%i] is : %s",msg_itr, some_data.some_text);
            }else{
                sprintf(some_data.some_text, "Message [%i] is sent.\n", msg_itr
                printf("%s", some_data.some_text);
            }
        if (msgsnd(msgid, (void *)&some_data, MAX_TEXT, 0) == -1){
            fprintf(stderr, "msgsnd failed\n");
            exit(EXIT_FAILURE);
        }
    }
    exit(EXIT_SUCCESS);
}
```

reciever.c

```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>
#include <sys/msg.h>

struct my_msg_st
{
    long int my_msg_type;
    char some_text[BUFSIZ];
};
int main(){
    int running = 1;
    int msgid;
    struct my_msg_st some_data;
    long int msg_to_receive = 0;
    key_t key = 1234;

    msgid = msgget(key, 0666 | IPC_CREAT);
    printf("msqid : %d \n", msgid);
    printf("key   : 0x%x \n", key);
    if (msgid == -1){
        fprintf(stderr, "msgget failed with error: %d\n", errno);
        exit(EXIT_FAILURE);
    }
    while (running){
        if (msgrcv(msgid, (void *)&some_data, BUFSIZ, msg_to_receive, 0) == -
            fprintf(stderr, "msgrcv failed with error: %d\n", errno);
            exit(EXIT_FAILURE);
        }
        printf("You wrote: %s", some_data.some_text);
        if (strncmp(some_data.some_text, "end", 3) == 0){
            running = 0;
        }
    }
    if (msgctl(msgid, IPC_RMID, 0) == -1){
        fprintf(stderr, "msgctl(IPC_RMID) failed\n");
        exit(EXIT_FAILURE);
    }
    exit(EXIT_SUCCESS);
}
```

First compile your c files to get *rmsg* and *smsg*:

1. *gcc reciever.c -o rmsg*
2. *gcc sender.c -o smsg*

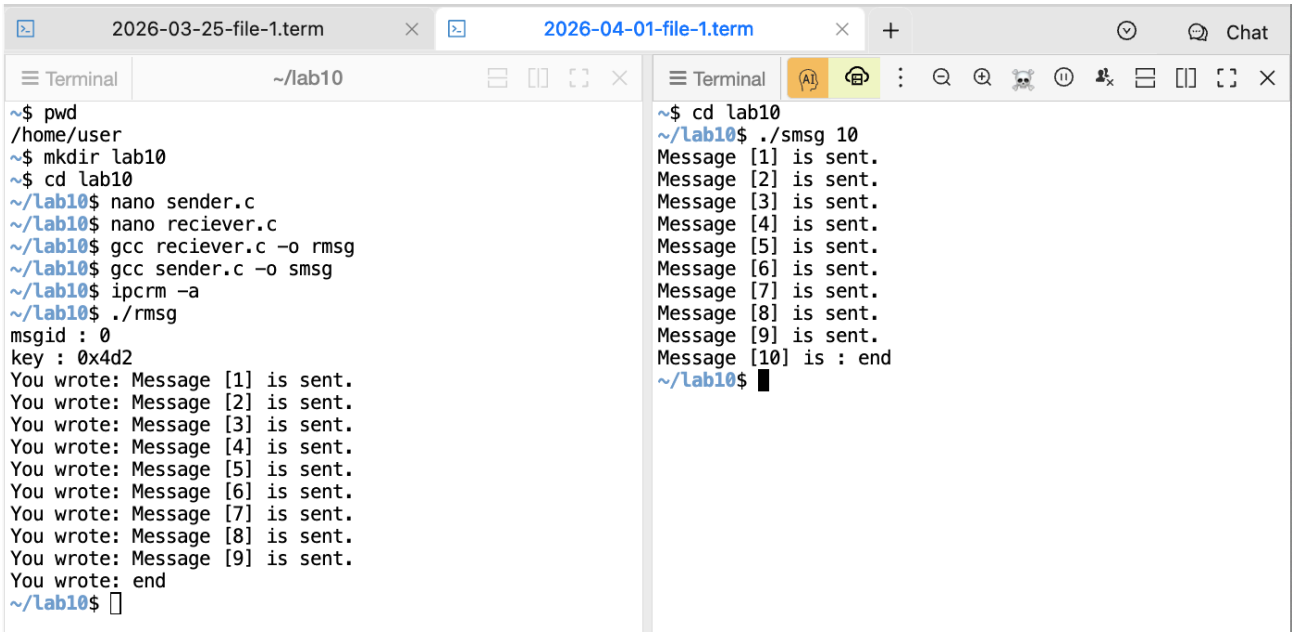
<u>Attention: Before answering the questions run the following command in your terminal to remove all existing messages:</u>

ipcrm -a

Use the following command and send 10 messages to the receiver.

./smsg 10

Insert a screenshot of your output.



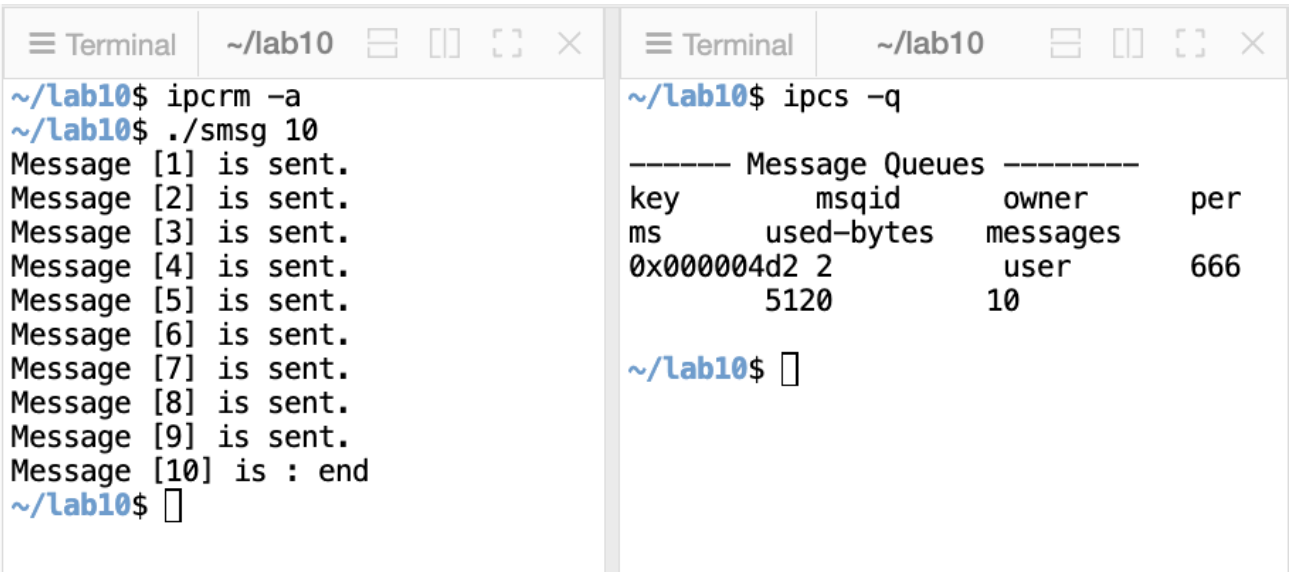
Screenshot 2026-04-01 at 11.40.54 AM.png (151.38 KB)

The correct answer is not displayed for Written Response type questions.

Question 2

Use *ipcs –q* to get the number of sent messages stored in the message queues.

Insert a screenshot of your output.



Screenshot 2026-04-01 at 11.46.29 AM.png (89.26 KB)

Screenshot 2026-04-01 at 11.50.38 AM.png (64.28 KB)

The correct answer is not displayed for Written Response type questions.

Question 3

✔ __5__ In Linux System V IPC, the option *-q* in the *ipcs* command refers to

✔ __8__ What does the command *ipcs -q* do?

1. Creates a new message queue
2. POSIX message queues only
3. Queues used for process scheduling
4. Displays information about shared memory segments
5. System V message queues currently present in the system
6. Deletes all IPC objects
7. Network packet queues
8. Displays information about message queues

Question 4

In order to retrieve your sent messages, use the following command:

./rmsg

Insert a screenshot of your output.

```
~/lab10$ ./rmsg
msgid : 2
key : 0x4d2
You wrote: Message [1] is sent.
You wrote: Message [2] is sent.
You wrote: Message [3] is sent.
You wrote: Message [4] is sent.
You wrote: Message [5] is sent.
You wrote: Message [6] is sent.
You wrote: Message [7] is sent.
You wrote: Message [8] is sent.
You wrote: Message [9] is sent.
You wrote: end
~/lab10$
```

Screenshot 2026-04-01 at 11.50.43 AM.png (81.59 KB)

The correct answer is not displayed for Written Response type questions.

Question 5

Referring to your results from Question 4, did you receive all your sent messages?

- ○ No
- ✓ ◉ Yes

Question 6

Referring to your results from Question 4 are there any messages left in the message queues (*ipcs –q*)?

- ✓ ◉ No

○ Yes

Question 7

Referring to your results from Question 6 (*ipcs –q*), insert a screenshot of your output.

```
~/lab10$ ipcs -q

------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages

~/lab10$
```

Screenshot 2026-04-01 at 11.54.14 AM.png (20.46 KB)

The correct answer is not displayed for Written Response type questions.

Question 8

Process communication using mmap. Run the code given below (https://cocalc.com/) and insert a screenshot of your output.

memap.c

```c
#include <unistd.h>
#include <string.h>
#include <sys/mman.h>
#include <stdio.h>
#include <sys/wait.h>

char ptype[10];
int main()
{
    int size = 50 * sizeof(int);
    void *addr = mmap(0, size, PROT_READ | PROT_WRITE, MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    printf("Mapped at : %p \n\n", addr);

    int *shared = addr;
    pid_t fork_return = fork();
    if (fork_return > 0)
    {
        shared[0] = 40;
        shared[1] = -20;
        strcpy(ptype, "Parent");
        int status;
        waitpid(-1, &status, 0);
    }
    else
    {
        sleep(1);
        printf("Child  : shared[0] = %d  , shared[1] =  %d  \n", shared[0], shared[1]);
        shared[1] = 120;
        strcpy(ptype, "Child ");
    }
    printf("%s : shared[0] : %d\n", ptype, shared[0]);
    printf("%s : shared[1] : %d\n", ptype, shared[1]);
    munmap(addr, size);
    return 0;
}
```

```
~/lab10$ gcc memap.c -o memap
~/lab10$ ./memap
Mapped at : 0x7fd8d2415000

Child : shared[0] = 40 , shared[1] = -20
Child : shared[0] : 40
Child : shared[1] : 120
Parent : shared[0] : 40
Parent : shared[1] : 120
~/lab10$
```

Screenshot 2026-04-01 at 11.55.58 AM.png (33.65 KB)

The correct answer is not displayed for Written Response type questions.

Question 9

What is the primary purpose of the **mmap()** system call in Unix/Linux systems?

- To perform inter-process communication using sockets
- To allocate heap memory dynamically
- To create a new process
- ✔ To map a file or device into a process's virtual address space

Question 10

Which of the following is a possible consequence of using **mmap()** with the **MAP_SHARED** flag?

- Changes to the mapped memory are visible only to the calling process
- The mapped region becomes read-only
- ✔ Changes to the mapped memory may be written back to the file and visible to other processes
- The mapping is automatically deallocated when fork() is called

Question 11

Referring to your results from Question 8 , do you think that the allocated memory was shared between the child and parent process.

- No
- ✔ Yes

Question 12

Referring to your results from Question 8, determine the final values of **`shared[0]`** and **`shared[1]`** in both the child and parent processes.

✔ __2__ Child (shared[0],shared[1])

✔ __2__ Parent (shared[0],shared[1])

1. 120, 40
2. 40, 120
3. 40, -20
4. 120, -20
5. -20, 120
6. -20, 40

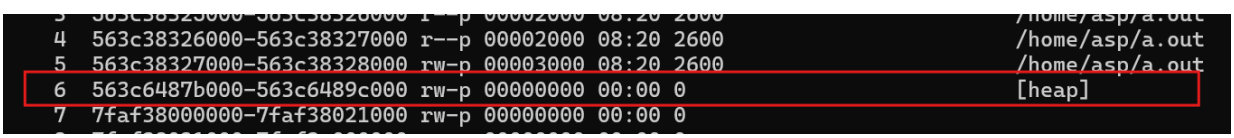
Question 13

Referring to your results from Question 8, use the following command to obtain the address space for the parent process.

./a.out & cat -n /proc/$!/maps

Highlight the location of mapped memory on the parent's address space and <u>insert</u> a screenshot of your output.

e.g. the location of the heap memory is highlighted.



```
~/lab10$ ./memap & cat -n /proc/$!/maps
[1] 958
Mapped at : 0x7f3b089f3000

     1  55e50e596000-55e50e597000 r--p 00000000 00:1b2 22                       /home/user/lab10/memap
     2  55e50e597000-55e50e598000 r-xp 00001000 00:1b2 22                       /home/user/lab10/memap
     3  55e50e598000-55e50e599000 r--p 00002000 00:1b2 22                       /home/user/lab10/memap
     4  55e50e599000-55e50e59a000 r--p 00002000 00:1b2 22                       /home/user/lab10/memap
     5  55e50e59a000-55e50e59b000 rw-p 00003000 00:1b2 22                       /home/user/lab10/memap
     6  55e5405e2000-55e540603000 rw-p 00000000 00:00 0                         [heap]
     7  7f3b087aa000-7f3b087ad000 rw-p 00000000 00:00 0
     8  7f3b087ad000-7f3b087d5000 r--p 00000000 00:f0 1213228                   /lib/x86_64-linux-gnu/libc.so.6
     9  7f3b087d5000-7f3b0895d000 r-xp 00028000 00:f0 1213228                   /lib/x86_64-linux-gnu/libc.so.6
    10  7f3b0895d000-7f3b089ac000 r--p 001b0000 00:f0 1213228                   /lib/x86_64-linux-gnu/libc.so.6
    11  7f3b089ac000-7f3b089b0000 r--p 001fe000 00:f0 1213228                   /lib/x86_64-linux-gnu/libc.so.6
    12  7f3b089b0000-7f3b089b2000 rw-p 00202000 00:f0 1213228                   /lib/x86_64-linux-gnu/libc.so.6
    13  7f3b089b2000-7f3b089bf000 rw-p 00000000 00:00 0
    14  7f3b089f3000-7f3b089f4000 rw-s 00000000 00:01 7944                      /dev/zero (deleted)
    15  7f3b089f4000-7f3b089f6000 rw-p 00000000 00:00 0
    16  7f3b089f6000-7f3b089f7000 r--p 00000000 00:f0 1212023                   /lib/x86_64-linux-gnu/ld-linux-x
86-64.so.2
    17  7f3b089f7000-7f3b08a22000 r-xp 00001000 00:f0 1212023                   /lib/x86_64-linux-gnu/ld-linux-x
86-64.so.2
    18  7f3b08a22000-7f3b08a2c000 r--p 0002c000 00:f0 1212023                   /lib/x86_64-linux-gnu/ld-linux-x
86-64.so.2
    19  7f3b08a2c000-7f3b08a2e000 r--p 00036000 00:f0 1212023                   /lib/x86_64-linux-gnu/ld-linux-x
86-64.so.2
    20  7f3b08a2e000-7f3b08a30000 rw-p 00038000 00:f0 1212023                   /lib/x86_64-linux-gnu/ld-linux-x
86-64.so.2
    21  7fffa24de000-7fffa2500000 rw-p 00000000 00:00 0                         [stack]
    22  7fffa2536000-7fffa253a000 r--p 00000000 00:00 0                         [vvar]
    23  7fffa253a000-7fffa253c000 r-xp 00000000 00:00 0                         [vdso]
    24  ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0                 [vsyscall]
```

Screenshot 2026-04-01 at 12.03.59 PM.png (194 KB)

The correct answer is not displayed for Written Response type questions.

Question 14

Use the Linux command *mknod* to create a named pipe with the name of MYFIFO.

Part A

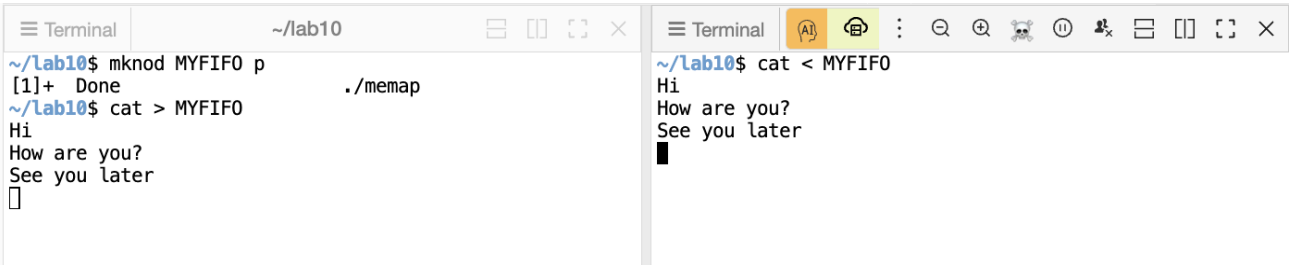
Open two Linux terminals side by side and run the following commands.

a. Terminal 1: run *cat > MYFIFO* and then send the following messages
Hi ¿
How are you? ¿
See you later ¿

b. Terminal 2: *cat < MYFIFO*

Hint: *¿* means to press the Enter key.

Ins Insert a screenshot of your output, ensuring that both terminals are included in the screenshot.



Screenshot 2026-04-01 at 12.02.17 PM.png (46.19 KB)

The correct answer is not displayed for Written Response type questions.

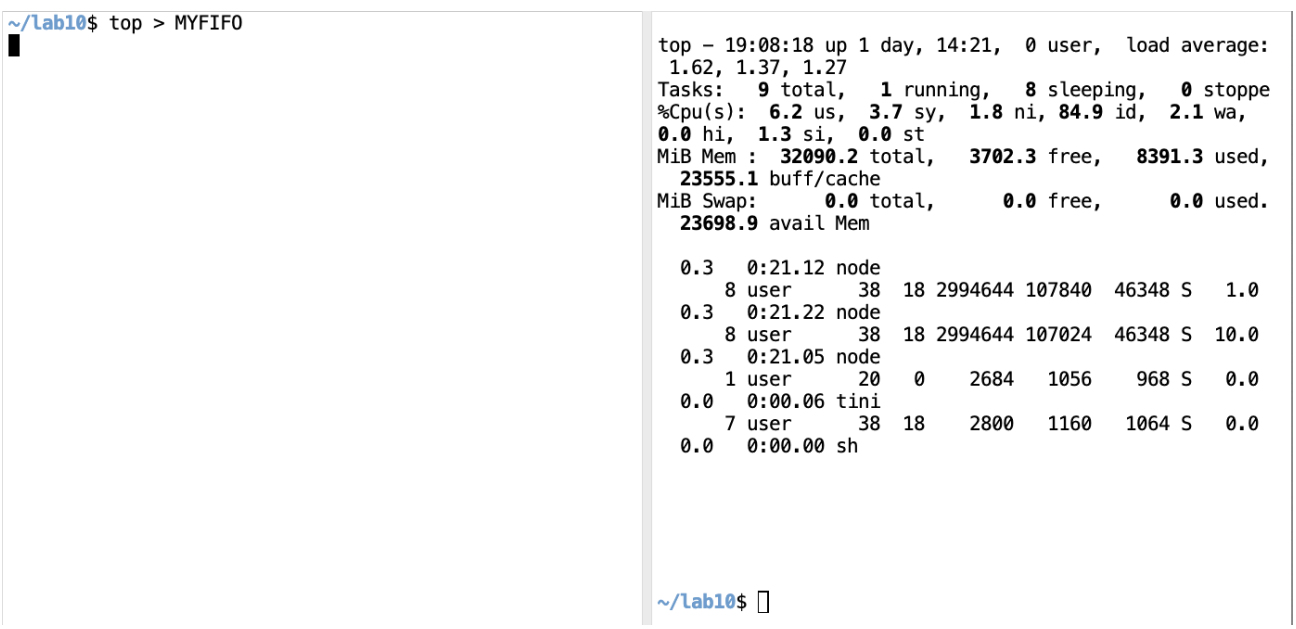
Question 15

Part B

Referring to your results from Question 14, again open two Linux terminals side by side and run the following commands.

a. Terminal 1: *top > MYFIFO*
b. Terminal 2: *cat < MYFIFO*

Insert a screenshot of your output, ensuring that both terminals are included in the screenshot.



Screenshot 2026-04-01 at 12.09.33 PM.png (97.85 KB)

The correct answer is not displayed for Written Response type questions.

Question 16

Which statement best describes a named pipe (FIFO) in Unix/Linux systems?

- It supports random access to data like a regular file
- ✔ It allows unrelated processes to communicate using a file-system name
- It allows communication only between parent and child processes

Question 17

What happens if a process opens a named pipe for reading and no process has opened it for writing?

- The named pipe is automatically deleted
- The read operation returns garbage data
- ✔ The open operation blocks until a writer opens the pipe
- The open operation immediately fails

Question 18

Answer the following questions after compiling and running the code.

sig.c

```
1  #include<stdio.h>
2  #include<signal.h>
3
4  void user_handler(int signum){
5      printf("user_handler : line %d \n", __LINE__);;
6  }
7
8  int main(){
9      signal(SIGUSR1,user_handler);
10     printf("main : line number %d \n", __LINE__);
11  //   raise(SIGUSR1);
12     printf("main : line number %d \n", __LINE__);
13     return 0;
14 }
```

Insert a screenshot of your output.

```
~/lab10$ gcc sig.c -o sig
~/lab10$ ./sig
main : line number 10
main : line number 12
~/lab10$
```

Screenshot 2026-04-01 at 12.12.08 PM.png (18.39 KB)

The correct answer is not displayed for Written Response type questions.

Question 19

Referring to your results from Question 18, uncomment line 11, recompile your code and run it. Insert a screenshot of your new output.

```
~/lab10$ gcc sig.c -o sig
~/lab10$ ./sig
main : line number 10
user_handler : line 5
main : line number 12
~/lab10$
```

Screenshot 2026-04-01 at 12.12.59 PM.png (20.28 KB)

The correct answer is not displayed for Written Response type questions.

Question 20

Which of the following statements about SIGUSR1 is correct?

- It always terminates the receiving process
- Its behavior is predefined and cannot be changed
- ✔ It can be caught, ignored, or handled by a user-defined signal handler
- It is generated only by the kernel

Question 21

If a process has installed a signal handler for SIGUSR1, what happens when raise(SIGUSR1) is executed??

- The kernel reboots the system
- ✔ The user-defined signal handler is executed
- The process ignores the signal
- The process is terminated immediately

Question 22

Is raise() considered a system call?

- ✔ No, it is a user-level library function, not a system call
- No, because signals are handled only in user space
- Yes, because it sends a signal

Yes, because it interacts with the kernel directly

Done